

INFORME DE ESPECIFICACIÓN DE DATOS Y BASE DE DATOS

Plataforma Web de Intermediación de Servicios u Oficios Informales

Elemento	Especificación
Backend	NestJS — API RESTful
ORM	Prisma
Base de datos	MySQL 8.0+
Frontend	Angular 17+
Autenticación	JWT + Bcrypt
Validación	class-validator + ValidationPipe
Documentación	Swagger / OpenAPI
Versión	2.0 — Migración integral a MySQL — Agosto 2026

Objetivo: actualizar integralmente el informe anterior para MySQL, conservando todo su contenido y adaptando tipos, relaciones, índices, claves, migraciones y Prisma.

Revisión técnica: MySQL no es solo un cambio de nombre. Se ajustan UUID, ENUM, BOOLEAN/TINYINT, DECIMAL, DATETIME, InnoDB, utf8mb4 y las relaciones para que el diseño sea coherente.

Índice

1. Resumen ejecutivo
2. Alcance
3. Arquitectura de datos
4. Modelo conceptual
5. Diccionario de datos
6. Relaciones
7. Integridad y reglas de negocio
8. Seguridad
9. Consultas y filtros
10. Índices y rendimiento
11. API REST
12. Estructura NestJS
13. Distribución Backend 1 / Backend 2
14. Prisma para MySQL
15. SQL MySQL de referencia
16. Migraciones y seed
17. Fuera de alcance
18. Extensiones futuras
19. Casos de uso
20. Checklist
21. Conclusiones

1. Resumen ejecutivo

La plataforma conecta clientes que necesitan servicios con trabajadores independientes que ofrecen oficios. Facilita descubrimiento, publicación, contacto y reputación, sin pagos ni chat interno.

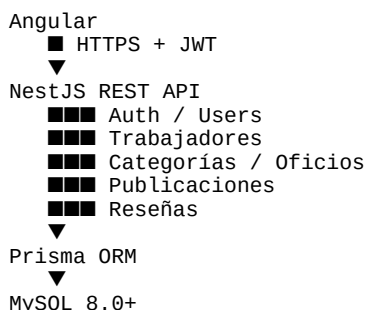
El núcleo es: usuarios, perfiles de trabajadores, categorías, oficios, trabajador_oficios, publicaciones, reseñas y sesiones. Se establece MySQL 8.0+ como motor.

2. Alcance

- Usuarios y autenticación.
- Roles CLIENTE y TRABAJADOR.
- Perfil profesional, experiencia, disponibilidad y zona.
- Hasta 3 oficios principales por trabajador.
- Categorías y oficios.
- Publicaciones y estados ABIERTA, EN_PROGRESO, COMPLETADA.
- Presupuesto opcional.
- Reseñas 1-5.
- WhatsApp y llamadas.
- Refresh tokens opcionales para sesiones revocables.

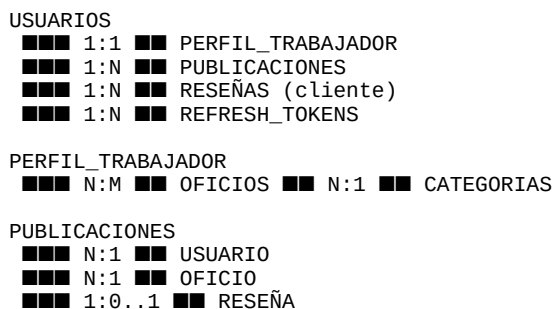
Fuera del alcance: pagos, tarjetas, chat, mensajes internos, comisiones, facturación, PWA/offline y sincronización local.

3. Arquitectura general



Angular nunca accede directamente a MySQL. Controllers reciben peticiones; services contienen reglas; Prisma gestiona persistencia.

4. Modelo conceptual



La relación N:M se resuelve con trabajador_oficios, incluyendo el indicador principal para controlar los tres oficios principales.

5. Diccionario de datos

5.1 usuarios

Campo	MySQL	Nulo	Regla	Descripción
id	CHAR(36)	No	PK	UUID textual.
nombre	VARCHAR(100)	No	—	Nombre.
apellido	VARCHAR(100)	No	—	Apellido.
email	VARCHAR(150)	No	UNIQUE	Correo.
telefono	VARCHAR(20)	No	UNIQUE	Teléfono.
password_hash	VARCHAR(255)	No	—	Hash Bcrypt.
rol	ENUM	No	CLIENTE/TRABAJADOR	Rol.
foto_url	TEXT	Sí	—	Foto.
activo	BOOLEAN	No	TRUE	Cuenta.
created_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Creación.
updated_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Actualización.

Se recomienda CHAR(36) para UUID. En alto volumen puede evaluarse BINARY(16). Email y teléfono son únicos.

5.2 perfiles_trabajador

Campo	MySQL	Nulo	Regla	Descripción
id	CHAR(36)	No	PK	Identificador.
usuario_id	CHAR(36)	No	UNIQUE/FK	Usuario.
descripcion	TEXT	Sí	—	Presentación.
experiencia	TEXT	Sí	—	Experiencia.
zona_cobertura	VARCHAR(150)	No	—	Zona.
disponible	BOOLEAN	No	TRUE	Disponibilidad.
created_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Creación.
updated_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Actualización.

5.3 categorias

Campo	MySQL	Nulo	Regla	Descripción
id	CHAR(36)	No	PK	Identificador.
nombre	VARCHAR(100)	No	UNIQUE	Nombre.
descripcion	TEXT	Sí	—	Descripción.
activo	BOOLEAN	No	TRUE	Estado.
created_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Creación.

5.4 oficios

Campo	MySQL	Nulo	Regla	Descripción
id	CHAR(36)	No	PK	Identificador.
categoria_id	CHAR(36)	No	FK	Categoría.
nombre	VARCHAR(100)	No	UNIQUE	Nombre.
descripcion	TEXT	Sí	—	Descripción.
activo	BOOLEAN	No	TRUE	Estado.
created_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Creación.

5.5 trabajador_oficios

Campo	MySQL	Nulo	Regla	Descripción
trabajador_id	CHAR(36)	No	PK/FK	Perfil.
oficio_id	CHAR(36)	No	PK/FK	Oficio.
principal	BOOLEAN	No	FALSE	Principal.
created_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Asociación.

PK compuesta: (trabajador_id, oficio_id).

5.6 publicaciones

Campo	MySQL	Nulo	Regla	Descripción
id	CHAR(36)	No	PK	Identificador.
cliente_id	CHAR(36)	No	FK	Cliente.
oficio_id	CHAR(36)	No	FK	Oficio.
titulo	VARCHAR(150)	No	—	Título.
descripcion	TEXT	No	—	Detalle.
ubicacion	VARCHAR(200)	No	—	Lugar.
presupuesto	DECIMAL(10,2)	Sí	>=0	Presupuesto.
estado	ENUM	No	3 estados	Ciclo.
created_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Creación.
updated_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Actualización.

cliente_id se obtiene del JWT; una publicación inicia ABIERTA.

5.7 reseñas

Campo	MySQL	Nulo	Regla	Descripción
id	CHAR(36)	No	PK	Identificador.
cliente_id	CHAR(36)	No	FK	Autor.
trabajador_id	CHAR(36)	No	FK	Evaluated.
publicacion_id	CHAR(36)	No	UNIQUE/FK	Trabajo.
calificacion	TINYINT UNSIGNED	No	1..5	Puntuación.
comentario	TEXT	Sí	—	Comentario.
created_at	DATETIME(3)	No	CURRENT_TIMESTAMP(3)	Fecha.

UNIQUE(publicacion_id) permite una reseña por publicación.

5.8 refresh_tokens

Campo	MySQL	Nulo	Descripción
id	CHAR(36)	No	Identificador.
usuario_id	CHAR(36)	No	Usuario.
token_hash	VARCHAR(255)	No	Hash.
expires_at	DATETIME(3)	No	Expiración.
revoked_at	DATETIME(3)	Sí	Revocación.
created_at	DATETIME(3)	No	Creación.

6. Relaciones y cardinalidades

Relación	Cardinalidad	Explicación
Usuario → Perfil	1:1	Un trabajador posee un perfil.
Categoría → Oficios	1:N	Una categoría agrupa oficios.
Trabajador → Oficios	N:M	Tabla puente.
Cliente → Publicaciones	1:N	Varias solicitudes.
Oficio → Publicaciones	1:N	Muchas solicitudes.
Cliente → Reseñas	1:N	Varias evaluaciones.
Trabajador → Reseñas	1:N	Evaluaciones recibidas.
Publicación → Reseña	1:0..1	Una reseña recomendada.
Usuario → RefreshTokens	1:N	Varias sesiones.

Decisión sobre contratación

Los requisitos no definen asignación formal. No se agrega una contratación ficticia. Si posteriormente se necesita saber quién tomó un trabajo, crear asignaciones o servicios_contratados.

7. Integridad y reglas

ID	Regla
RN-01	Un usuario tiene un único rol.
RN-02	Solo trabajadores tienen perfil.
RN-03	Máximo 3 oficios principales.
RN-04	Solo clientes crean publicaciones.
RN-05	Publicación ligada a oficio activo.
RN-06	Nueva publicación = ABIERTA.
RN-07	Solo propietario modifica.
RN-08	COMPLETADA no se edita libremente.
RN-09	Calificación 1..5.
RN-10	Cliente no puede evaluarse a sí mismo.
RN-11	Identidad del JWT.
RN-12	Password con Bcrypt.
RN-13	NULL presupuesto = no especificado.
RN-14	Promedio con AVG().
RN-15	Preferir baja lógica en catálogos.

8. Seguridad

Registro: DTO → unicidad → Bcrypt → Usuario → Perfil → JWT
Login: credenciales → Bcrypt → JWT
Petición: JWT → JwtAuthGuard → RolesGuard → Controller
→ Service → Prisma → MySQL

Usar ValidationPipe global, class-validator, Guards y comprobación de propiedad. Nunca devolver password_hash.

9. Consultas y filtros

Consulta	Filtros
Trabajadores	oficioid, categoriald, zona, disponible
Publicaciones	oficioid, categoriald, ubicación, estado
Feed	estado, oficio, ubicación, paginación
Perfil	trabajadorId
Reputación	trabajadorId
Mis publicaciones	clienteId desde JWT

10. Índices y rendimiento

Tabla	Índices
usuarios	UNIQUE(email), UNIQUE(telefono), INDEX(rol)
perfiles_trabajador	INDEX(zona_cobertura), INDEX(disponible)
categorias	UNIQUE(nombre), INDEX(activo)
oficios	UNIQUE(nombre), INDEX(categoria_id), INDEX(activo)
trabajador_oficios	INDEX(oficio_id), INDEX(trabajador_id)
publicaciones	INDEX(cliente_id), INDEX(oficio_id), INDEX(estado), INDEX(ubicacion), INDEX(created_at)
resenas	INDEX(trabajador_id), INDEX(cliente_id, publicacion_id)
refresh_tokens	INDEX(usuario_id), INDEX(expires_at)

El objetivo <300 ms depende de hosting, red, volumen, consultas y paginación. Los índices ayudan, pero debe medirse con datos reales.

11. API REST

Auth

Método	Endpoint	Descripción	Acceso
POST	/auth/register	Crear cuenta	Público
POST	/auth/login	Iniciar sesión	Público
POST	/auth/refresh	Renovar	Token
POST	/auth/logout	Revocar	Autenticado

Usuarios / trabajadores

Método	Endpoint	Descripción
GET	/users/me	Perfil
PATCH	/users/me	Editar datos
GET	/trabajadores	Buscar
GET	/trabajadores/:id	Perfil público
PATCH	/trabajadores/me	Editar profesional
PUT	/trabajadores/me/oficios	Gestionar oficios

Catálogo / publicaciones / reseñas

Método	Endpoint	Descripción
GET	/categorias	Categorías
GET	/oficios	Oficios
POST	/publicaciones	Crear
GET	/publicaciones	Feed
GET	/publicaciones/:id	Detalle
PATCH	/publicaciones/:id	Editar/estado
DELETE	/publicaciones/:id	Eliminar propia
POST	/resenas	Crear

Método	Endpoint	Descripción
GET	/trabajadores/:id/resenas	Reseñas
GET	/trabajadores/:id/resumen	Resumen

12. Estructura NestJS

```

src/
├── auth/
├── users/
├── trabajadores/
├── categorias/
├── oficios/
├── publicaciones/
├── resenas/
├── prisma/
│   ├── prisma.service.ts
│   ├── prisma.module.ts
│   └── common/
│       ├── decorators/
│       ├── guards/
│       ├── filters/
│       └── pipes/
├── app.module.ts
└── main.ts

```

Cada dominio debe tener controller, service, module y DTOs. La lógica permanece en services.

13. Distribución Backend 1 / Backend 2

Equipo	Responsabilidades	Tablas
Backend 1 — Core & Auth	NestJS, Prisma, migraciones, JWT, Bcrypt, usuarios, perfiles, roles, Swagger	usuarios, perfiles_trabajador, refresh_tokens
Backend 2 — Negocio	categorías, oficios, publicaciones, filtros, reseñas	categorias, oficios, trabajador_oficios, publicaciones, resenas

El contrato compartido debe incluir schema.prisma, enums, DTOs, JWT, errores, paginación y respuestas.

14. Modelo Prisma para MySQL

```
datasource db { provider = "mysql"; url = env("DATABASE_URL") }
generator client { provider = "prisma-client-js" }

enum Role { CLIENTE TRABAJADOR }
enum PublicacionEstado { ABIERTA EN_PROGRESO COMPLETADA }

model Usuario {
  id String @id @default(uuid()) @db.Char(36)
  nombre String @db.VarChar(100); apellido String @db.VarChar(100)
  email String @unique @db.VarChar(150); telefono String @unique @db.VarChar(20)
  passwordHash String @db.VarChar(255); rol Role
  fotoUrl String? @db.Text; activo Boolean @default(true)
  createdAt DateTime @default(now()) @db.DateTime(3); updatedAt DateTime @updatedAt @db.DateTime(3)
  perfilTrabajador PerfilTrabajador?; publicaciones Publicacion[]
  resenasComoCliente Resena[] @relation("ResenasCliente")
  resenasComoTrabajador Resena[] @relation("ResenasTrabajador")
  refreshTokens RefreshToken[]
  @@index([rol])
}

model PerfilTrabajador {
  id String @id @default(uuid()) @db.Char(36); usuarioId String @unique @db.Char(36)
  descripcion String? @db.Text; experiencia String? @db.Text
  zonaCobertura String @db.VarChar(150); disponible Boolean @default(true)
  createdAt DateTime @default(now()) @db.DateTime(3); updatedAt DateTime @updatedAt @db.DateTime(3)
  usuario Usuario @relation(fields:[usuarioId], references:[id], onDelete:Cascade)
  oficios TrabajadorOficio[]
  @@index([zonaCobertura]); @@index([disponible])
}

model Categoria {
  id String @id @default(uuid()) @db.Char(36); nombre String @unique @db.VarChar(100)
  descripcion String? @db.Text; activo Boolean @default(true)
  createdAt DateTime @default(now()) @db.DateTime(3); oficios Oficio[]
  @@index([activo])
}

model Oficio {
  id String @id @default(uuid()) @db.Char(36); categoriaId String @db.Char(36)
  nombre String @unique @db.VarChar(100); descripcion String? @db.Text
  activo Boolean @default(true); createdAt DateTime @default(now()) @db.DateTime(3)
  categoria Categoria @relation(fields:[categoriaId], references:[id], onDelete:Restrict)
  trabajadores TrabajadorOficio[]; publicaciones Publicacion[]
  @@index([categoriaId]); @@index([activo])
}

model TrabajadorOficio {
  trabajadorId String @db.Char(36); oficioId String @db.Char(36)
  principal Boolean @default(false); createdAt DateTime @default(now()) @db.DateTime(3)
  trabajador PerfilTrabajador @relation(fields:[trabajadorId], references:[id], onDelete:Cascade)
  oficio Oficio @relation(fields:[oficioId], references:[id], onDelete:Restrict)
  @@id([trabajadorId, oficioId]); @@index([oficioId])
}

model Publicacion {
  id String @id @default(uuid()) @db.Char(36); clienteId String @db.Char(36); oficioId String @db.Char(36)
  titulo String @db.VarChar(150); descripcion String @db.Text; ubicacion String @db.VarChar(200)
  presupuesto Decimal? @db.Decimal(10,2); estado PublicacionEstado @default(ABIERTA)
  createdAt DateTime @default(now()) @db.DateTime(3); updatedAt DateTime @updatedAt @db.DateTime(3)
  cliente Usuario @relation(fields:[clienteId], references:[id], onDelete:Restrict)
  oficio Oficio @relation(fields:[oficioId], references:[id], onDelete:Restrict)
  resena Resena?
  @@index([clienteId]); @@index([oficioId]); @@index([estado]); @@index([ubicacion]); @@index([createdAt])
}

model Resena {
  id String @id @default(uuid()) @db.Char(36); clienteId String @db.Char(36); trabajadorId String @db.Char(36)
  publicacionId String @unique @db.Char(36); calificacion Int; comentario String? @db.Text
  createdAt DateTime @default(now()) @db.DateTime(3)
  cliente Usuario @relation("ResenasCliente", fields:[clienteId], references:[id], onDelete:Restrict)
  trabajador Usuario @relation("ResenasTrabajador", fields:[trabajadorId], references:[id], onDelete:Restrict)
  publicacion Publicacion @relation(fields:[publicacionId], references:[id], onDelete:Restrict)
  @@index([trabajadorId]); @@index([clienteId])
}

model RefreshToken {
  id String @id @default(uuid()) @db.Char(36); usuarioId String @db.Char(36)
  tokenHash String @db.VarChar(255); expiresAt DateTime @db.DateTime(3)
  revokedAt DateTime? @db.DateTime(3); createdAt DateTime @default(now()) @db.DateTime(3)
  usuario Usuario @relation(fields:[usuarioId], references:[id], onDelete:Cascade)
```

```

    @@index([usuarioId]); @@index([expiresAt])
  }

```

Nota: el bloque anterior es un esquema de referencia compacto para documentación; al implementarlo en Prisma, cada campo debe declararse en una línea válida de schema.prisma. MySQL usa InnoDB, utf8mb4, ENUM, DECIMAL y DATETIME(3).

15. SQL MySQL de referencia

```

CREATE DATABASE IF NOT EXISTS plataforma_oficios
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE plataforma_oficios;

```

```

-- Prisma será la fuente de verdad de tablas y migraciones.
-- Motor recomendado: InnoDB.
-- PK/FK: CHAR(36).
-- Presupuesto: DECIMAL(10,2).
-- Calificación: TINYINT UNSIGNED.
-- Fechas: DATETIME(3).

```

No se recomienda mezclar migraciones SQL manuales y Prisma sin una estrategia definida.

16. Migraciones y seed

```

npm install prisma @prisma/client
npx prisma init
npx prisma migrate dev --name init
npx prisma generate
npx prisma db seed

```

```

# Producción
npx prisma migrate deploy

```

El seed debe crear categorías/oficios y datos de prueba. Las contraseñas de seed también usan Bcrypt.

17. Fuera de alcance

Elemento	Tabla	Razón
Pagos	No	Acuerdo externo.
Tarjetas	No	Sin procesamiento financiero.
Chat	No	WhatsApp/teléfono.
Facturas	No	No solicitado.
Comisiones	No	Sin intermediación financiera.
Offline	No	Excluido.

18. Extensiones futuras

Entidad	Propósito
asignaciones	Trabajador que toma publicación.
favoritos	Guardar perfiles/publicaciones.
notificaciones	Avisos.
reportes	Moderación.
ubicaciones	Coordenadas/distancia.
media	Fotos/portafolio.
historial_estados	Auditoría.
verificaciones	Identidad/experiencia.

19. Casos de uso

Registro de trabajador

POST /auth/register → DTO → unicidad → Bcrypt → Usuario → PerfilTrabajador → tokens

Crear publicación

JWT → cliente → rol CLIENTE → DTO → oficio activo → Publicacion ABIERTA

Buscar trabajadores

GET /trabajadores → oficio/zona/disponibilidad → joins → paginación → datos públicos

Crear reseña

JWT → cliente → validar publicación → elegibilidad → Resena 1..5

20. Checklist

- MySQL 8.0+ definido.
- Prisma provider mysql.
- DATABASE_URL.
- schema.prisma + migración.
- UUID/enums revisados.
- InnoDB + utf8mb4.
- Seed.
- PrismaService.
- Bcrypt.
- JWT/Guards.
- Roles/propiedad.
- Perfil trabajador.
- trabajador_oficios.
- Máximo 3 principales.
- Publicaciones.
- Filtros/paginación.
- Estados.
- Reseñas.
- Promedio.
- Índices.
- Swagger.
- ValidationPipe.
- No exponer password_hash.
- Errores consistentes.
- Datos de prueba.
- Pruebas de FK.
- DTOs/respuestas documentados.

- Medición de consultas.
- Migraciones en CI/CD.

21. Conclusiones

El cambio de PostgreSQL a MySQL queda incorporado integralmente: MySQL 8.0+, InnoDB, utf8mb4, CHAR(36) para UUID, ENUM para estados/roles, DECIMAL(10,2) para presupuesto, TINYINT UNSIGNED para calificaciones y DATETIME(3) para fechas.

La arquitectura queda: Angular → HTTPS/JWT → NestJS → Controllers → Services → Prisma → MySQL. El Backend controla autenticación, roles, propiedad y reglas; Angular consume la API.

Se mantiene la decisión de no crear contratación hasta que el requisito la defina. Si posteriormente se necesita registrar qué trabajador tomó un servicio, se agrega una entidad de asignación.

Recomendación final: aprobar este modelo como contrato de datos antes del desarrollo paralelo. Cambios posteriores deben realizarse mediante migraciones versionadas de Prisma.